



المدرسة الوطنية للعلوم التطبيقية - بني ملال
ⵜⴰⴳⴷⴰⵢⵜ ⵜⴰⵎⴻⵔⴰⵏⵜ ⵜⴰⵖⴻⵔⴰⵏⵜ ⵜⴰⵎⴻⵔⴰⵏⵜ
Ecole Nationale des Sciences Appliquées - Béni Mellal

Module :

Génie Logiciel

Compte Rendu du TP 1 :
Programmation orientée objet (POO)

2^{ème} année : Filière Transformation Digitale Industrielle

Réalisé par :

ECH-CHOKHMANY Loubna

Professeur :

- Pr. BE.ELBAGHAZAOU

Année universitaire 2025 / 2026

I. Introduction :

Ce rapport présente les travaux réalisés dans le cadre du TP N°1 portant sur la Programmation Orientée Objet (POO) en Java. Les exercices proposés visent à illustrer les quatre piliers fondamentaux de la POO : l'encapsulation, l'héritage, l'abstraction et le polymorphisme.

Trois exercices structurés permettent de mettre en pratique ces concepts à travers des exemples concrets : un système de gestion d'un zoo, un système de gestion de véhicules, et un système de gestion de comptes bancaires.

II. Exercice 1.1 - Système de gestion d'un Zoo :

Partie 1 : Définitions théoriques

1. Encapsulation :

L'encapsulation est le mécanisme qui consiste à regrouper les données (attributs) et les comportements (méthodes) d'un objet au sein d'une même unité, tout en contrôlant l'accès à ces données depuis l'extérieur.

Dans la classe `Animal`, l'attribut `nom` est déclaré `private`, ce qui interdit tout accès direct depuis l'extérieur. L'accès se fait uniquement via la méthode publique `getNom()`. C'est l'essence même de l'encapsulation.

Exemple

```
private String nom; → accès uniquement via getNom()
```

2. Héritage :

L'héritage est un mécanisme permettant à une classe (sous-classe) de réutiliser les attributs et méthodes d'une autre classe (superclasse), tout en pouvant les étendre ou les redéfinir.

Dans le code, les classes `Mammifere` et `Oiseau` héritent toutes deux de la classe abstraite `Animal` via le mot-clé `extends`. Elles récupèrent ainsi l'attribut `nom` et la méthode `getNom()`, tout en fournissant leur propre implémentation de `faireDuBruit()`.

3. Abstraction :

L'abstraction consiste à définir une interface commune (méthodes) sans fournir leur implémentation concrète. En Java, elle se matérialise par les classes abstraites (abstract class) et les interfaces.

La classe Animal est déclarée abstract et contient la méthode abstraite faireDuBruit(). Il est ainsi impossible d'instancier directement un Animal : chaque sous-classe est forcée de fournir son propre comportement sonore.

4. Polymorphisme

Le polymorphisme permet à un objet d'être manipulé sous plusieurs formes. En Java, le polymorphisme d'inclusion (ou de sous-type) permet à une variable de type superclasse de référencer un objet de sous-classe.

Dans la méthode main(), les variables tigre et perroquet sont déclarées de type Animal, mais référencent respectivement un Mammifere et un Oiseau. L'appel à faireDuBruit() déclenche automatiquement la bonne implémentation selon le type réel de l'objet.

Partie 2 : Complétion du code

5. Méthode voler() dans la classe Oiseau :

La méthode voler() est décommentée et implémentée comme suit :

```
public void voler() {  
    System.out.println("L'oiseau vole.");  
}
```

6. Modification de main() pour utiliser voler() :

Pour appeler voler(), la variable perroquet doit être déclarée comme Oiseau (et non Animal), car la méthode voler() est spécifique à Oiseau et n'existe pas dans Animal :

```
Oiseau perroquet = new Oiseau("Perroquet");  
perroquet.faireDuBruit();  
perroquet.voler();
```

Partie 3 : Identification des mots-clés

Concept	Mot-clé Java	Exemple dans le code
Héritage	extends	class Mammifere extends Animal
Redéfinition de méthode	@Override	@Override public void faireDuBruit()
Abstraction	abstract	abstract class Animal / abstract void faireDuBruit()

III. Exercice 1.2 - Système de gestion des véhicules :

Partie 1 : Théorie et concepts

1. Encapsulation :

Dans la classe Vehicule, les trois attributs marque, modele et annee sont déclarés private. L'accès est uniquement possible via les méthodes getMarque(), getModele() et getAnnee(). Les sous-classes Voiture et Camion y accèdent via ces accesseurs, respectant ainsi le principe d'encapsulation.

2. Héritage :

La classe Voiture et la classe Camion héritent toutes deux de Vehicule grâce au mot-clé extends. Cela leur permet de réutiliser les attributs et méthodes de Vehicule (marque, modele, annee, et les getters associés), tout en ajoutant leurs propres attributs spécifiques (nombreDePortes pour Voiture, capaciteDeCharge pour Camion).

3. Polymorphisme :

Dans main(), maVoiture et monCamion sont déclarés de type Vehicule mais référencent respectivement une Voiture et un Camion. L'appel afficherDetails() invoque dynamiquement l'implémentation correcte selon le type réel de l'objet. C'est le polymorphisme d'exécution (late binding).

4. Abstraction :

La classe Vehicule est abstraite car elle contient la méthode abstraite afficherDetails(). On ne peut pas créer directement un objet Vehicule, car un véhicule générique sans type précis n'a pas de sens dans ce contexte. L'abstraction force les sous-classes à fournir une implémentation concrète adaptée à leur nature.

Partie 2 : Complétion du code

5. Méthode demarrer() et modification de afficherDetails() :

On ajoute une méthode concrète demarrer() dans Vehicule, et on l'appelle dans chaque sous-classe :

```
// Dans Vehicule :  
  
public void demarrer() {  
    System.out.println("Le véhicule démarre.");  
}  
  
// Dans Voiture.afficherDetails() :  
demarrer();  
System.out.println("Voiture: " + getMarque() + " ...");  
  
// Dans Camion.afficherDetails() :  
demarrer();  
System.out.println("Camion: " + getMarque() + " ...");
```

6. Nouvelle classe Moto :

```
class Moto extends Vehicule {  
    private String typeDeGuidon;  
  
    public Moto(String marque, String modele, int annee, String typeDeGuidon) {  
        super(marque, modele, annee);  
        this.typeDeGuidon = typeDeGuidon;}  
  
    @Override  
    public void afficherDetails() {  
        System.out.println("Moto: " + getMarque() + " " + getModele()  
            + " (" + getAnnee() + "), Guidon: " + typeDeGuidon);  
    }  
}
```

Partie 3 : Identification des mots-clés :

Concept	Mot-clé Java	Exemple
Redéfinition de méthode	@Override	@Override public void afficherDetails()
Héritage	extends	class Voiture extends Vehicule
Appel constructeur parent	super()	super(marque, modele, annee)

III. Exercice 1.3 - Système de gestion des comptes bancaires :

1. Description :

Cet exercice consiste à implémenter de zéro un système orienté objet de gestion de comptes bancaires. On distingue deux types de comptes : le CompteCourant et le CompteEpargne. Ces deux classes héritent d'une classe de base commune CompteBancaire.

2. Analyse des concepts POO utilisés :

Concept	Application dans l'exercice 1.3
Encapsulation	Attributs numeroCompte et solde déclarés private dans CompteBancaire, accessibles via getSolde() et getNumeroCompte().
Héritage	CompteEpargne et CompteCourant héritent de CompteBancaire via extends, réutilisant déposer() et retirer().
Polymorphisme	Les deux types de compte peuvent être manipulés comme des CompteBancaire (référence de type parent).
Abstraction	CompteBancaire regroupe les comportements communs, masquant la complexité à l'utilisateur final.

IV. Conclusion :

Ce TP a permis de mettre en pratique les quatre piliers de la Programmation Orientée Objet en Java à travers trois exercices progressifs. La gestion du zoo, des véhicules et des comptes bancaires ont illustré comment :

- L'encapsulation protège les données internes d'un objet en limitant leur accès direct.
- L'héritage favorise la réutilisation du code en établissant des relations hiérarchiques entre classes.
- L'abstraction permet de définir des contrats que les sous-classes doivent respecter.
- Le polymorphisme offre la flexibilité de traiter des objets de types différents de manière uniforme.